Understand, Then Modernize: AI and the Solution Architect in Banking Legacy

Wallace Espindola
Solution Architect

wallace.espindola@gmail.com
linkedin.com/in/wallaceespindola | github.com/wallaceespindola

The problem: knowledge buried in code

- Decades of **interest, fee, rounding, eligibility and reconciliation** rules
- Written down **only in the code**, known by a few specialists
- Batch cycles nobody fully remembers
- Data formats that break naive conversions
- Mainframes stay because they are **fast, stable and audited**

Rewrite before you understand, and you replace a reliable system with a less reliable one.

Why it is hard

Area	Typical difficulty
People	Scarce COBOL skills, rules known by few, org chart mirrors the monolith
Technology	Coupled programs, CICS/IMS, JCL and schedulers, dead and duplicated code
Integration	Payments, cards, risk, reporting; overnight batch next to real-time channels
Data	DB2, VSAM, `COMP-3`, encoding, dates, rounding, sync during coexistence
Risk and cost	Availability, DORA, double-run cost, a less reliable target

The data traps generic lists leave out

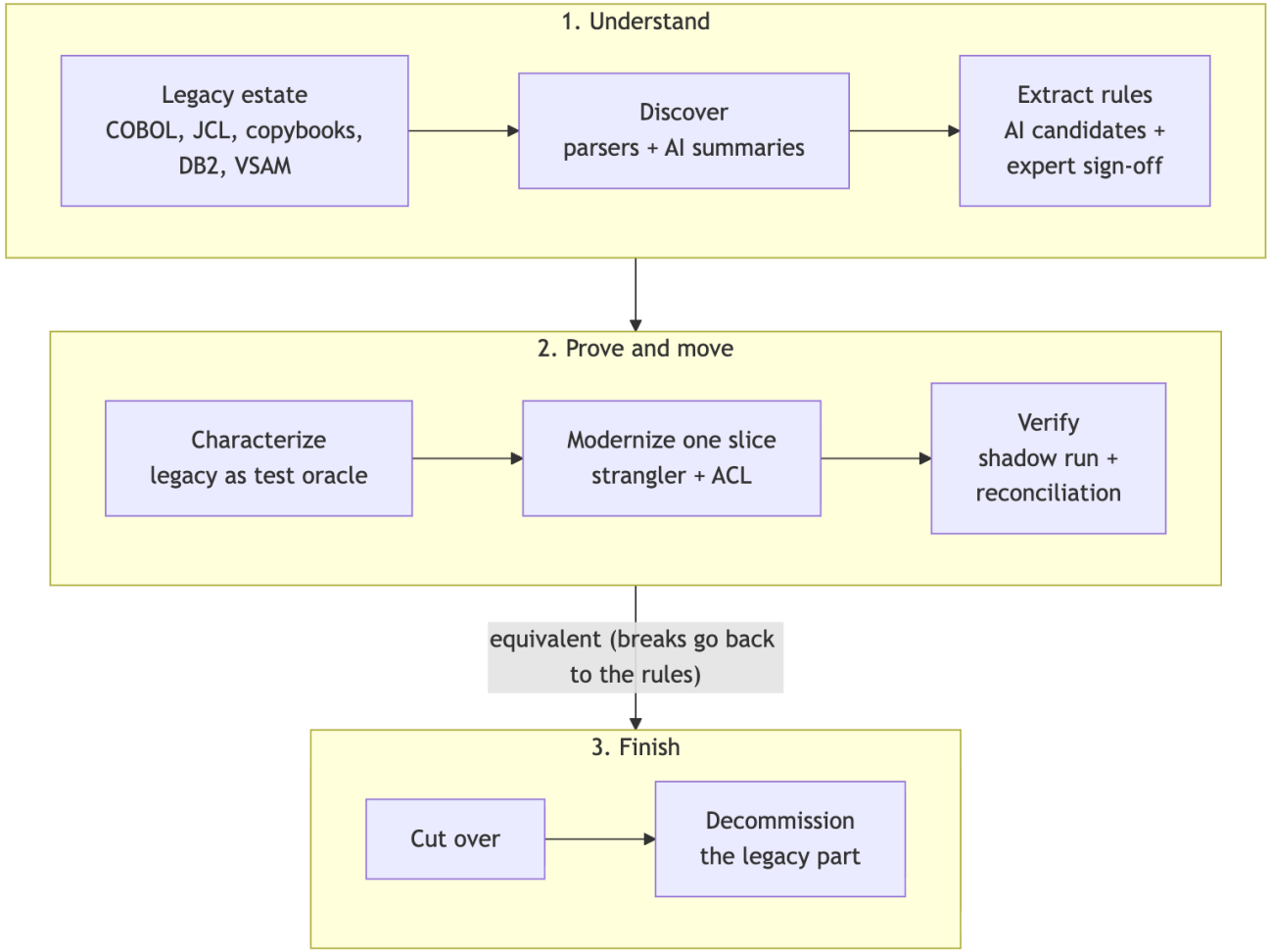
- `COMP-3` packed decimal, signed zoned decimal, `REDEFINES`, `OCCURS DEPENDING ON`
- Implied decimal points: `PIC 9(7)V99`
- EBCDIC collation is not ASCII sort order
- **COBOL truncates** unless `ROUNDED` is specified
- Julian dates, century windowing, business-day calendars
- Batch restart: checkpoints, end-of-day ordering, exactly once per business date

Target code uses `BigDecimal` with explicit scale and `RoundingMode`, never `double`.

Three principles

1. **Modernize capabilities, not code.** The unit of change is a business capability, not a program or a language.
2. **Correctness is proven, not assumed.** Characterization tests, parallel runs and ledger-level reconciliation decide when a slice is done.
3. **AI proposes, deterministic tooling verifies, humans approve.**

The modernization loop

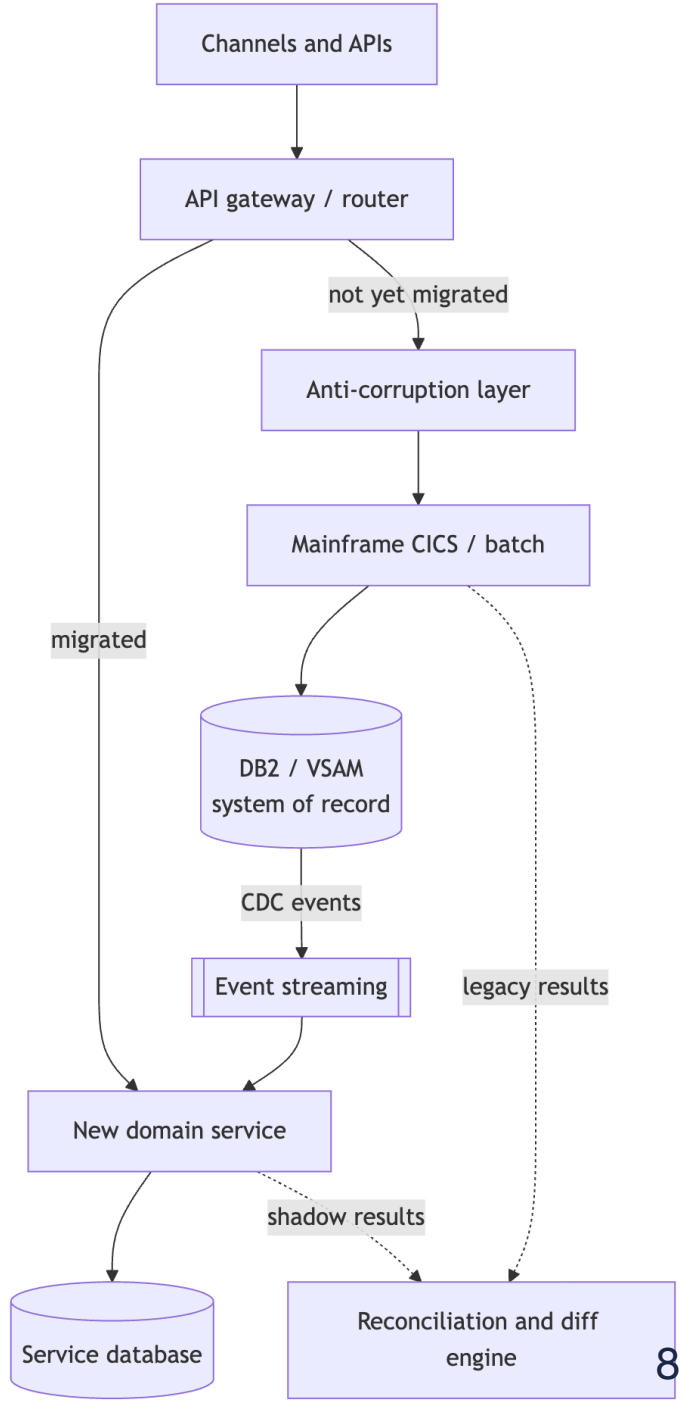


Strategy options, decided per capability

Strategy	Good fit	Main risk
Retain / encapsulate	Stable core ledger, low change rate	Skills risk and cost stay
Rehost / replatform	Hardware or licensing exit, low change	Same design, new ops complexity
Refactor (automated)	Large, well-tested estate, short timeline	"JOBOL", no maintainability gain
Rearchitect / rebuild	High-change, differentiating capability	Rule loss, long parallel run
Replace (package/SaaS)	Commodity capability	Customization creep, lock-in
Retire	Dead or duplicated code	Hidden consumers

Strangler with controlled coexistence

- **Router** switches traffic per capability, product or segment, with instant fallback
- **Anti-corruption layer** keeps legacy codes out of the new model
- **CDC, not dual writes:** one writer per data element
- **Shadow mode:** the new service computes, does not commit
- **One system of record** per business date, written down



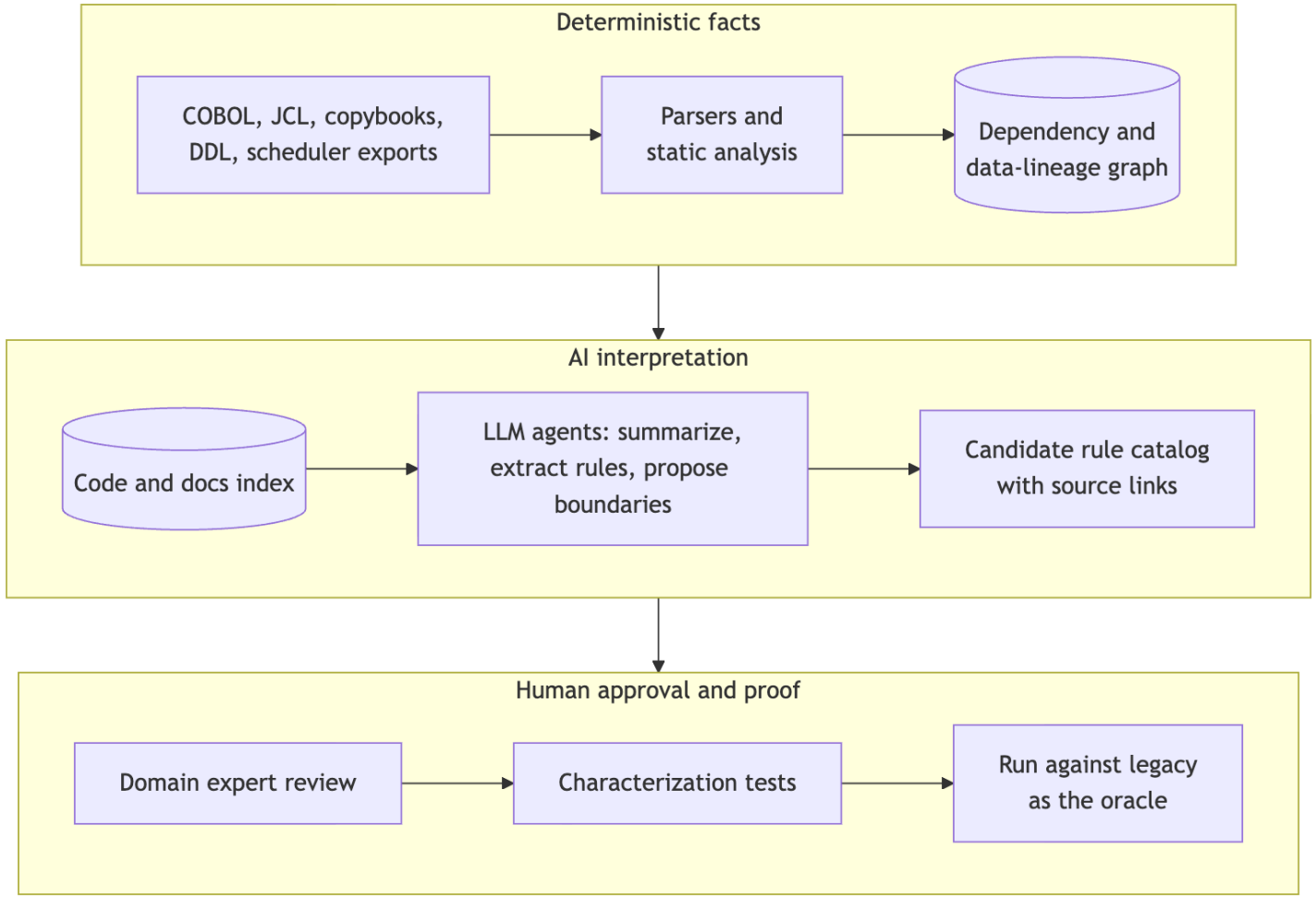
The architect's job

- Reconstruct the current state: applications, interfaces, jobs, data flows
- Define capabilities and domain boundaries **with domain experts**
- Set target and transition architectures
- Choose the **consistency model** per capability
- Set non-functional targets: end-of-day peaks, RPO/RTO, batch window
- Plan waves by value and risk; govern equivalence, reconciliation and rollback gates
- Record decisions as ADRs; keep the business case honest

Where AI helps, and the control for each

Area	AI contribution	Required control
Discovery	Summarize programs, copybooks, jobs	Ground on parser output; cite file and line
Rule extraction	Candidate rules with source links	Expert sign-off; rule catalog with status
Domain modeling	Candidate bounded contexts	Coupling metrics and workshops
Code modernization	Assist refactoring	Review, static analysis, tests pass
Testing	Characterization tests, discrepancy analysis	Legacy is the oracle, not the model
Operations	Explain abends and breaks	Read-only; no autonomous actions

Reference AI-assisted discovery pipeline



The guardrail

AI must not independently approve banking correctness

Deterministic parsers and static analysis first. Then human review, the legacy system as test oracle, financial reconciliation and secure model deployment.

One cent: the legacy rule

Rule BR-INT-001, monthly interest, from `examples/cobol/INTCALC.cbl`:

```
           IF ACCT-ACTIVE
       *       No ROUNDED phrase: the result is truncated to two decimals.
               COMPUTE WS-MONTHLY-INT =
                   ACCT-BALANCE * ACCT-ANNUAL-RATE / 1200
           ELSE
               MOVE ZERO TO WS-MONTHLY-INT
           END-IF.
```

No `ROUNDED`, so the result is **truncated toward zero** to two decimals.

Two Java ports, one of them wrong

From `InterestRule.java`:

```
// Legacy-equivalent: truncation toward zero, like COBOL without ROUNDED
return balance.multiply(annualRatePercent)
    .divide(MONTHS_TIMES_PERCENT, MONEY_SCALE, RoundingMode.DOWN);

// The port a reviewer would accept at first glance: banker's rounding
return balance.multiply(annualRatePercent)
    .divide(MONTHS_TIMES_PERCENT, MONEY_SCALE, RoundingMode.HALF_EVEN);
```

Both use `BigDecimal`. Only one matches the legacy system.

The golden master catches it: 3 of 8 accounts break

Account	Balance	Rate %	Legacy (DOWN)	Naive (HALF_EVEN)
ACC-0001	1000.00	5.00	**4.16**	4.17
ACC-0004	999999.99	4.10	**3416.66**	3416.67
ACC-0007	-100.00	19.99	**-1.66**	-1.67

The other 5 accounts match. Fixture: `monthly-interest.csv`, **illustrative, worked out by hand from the COBOL semantics.**

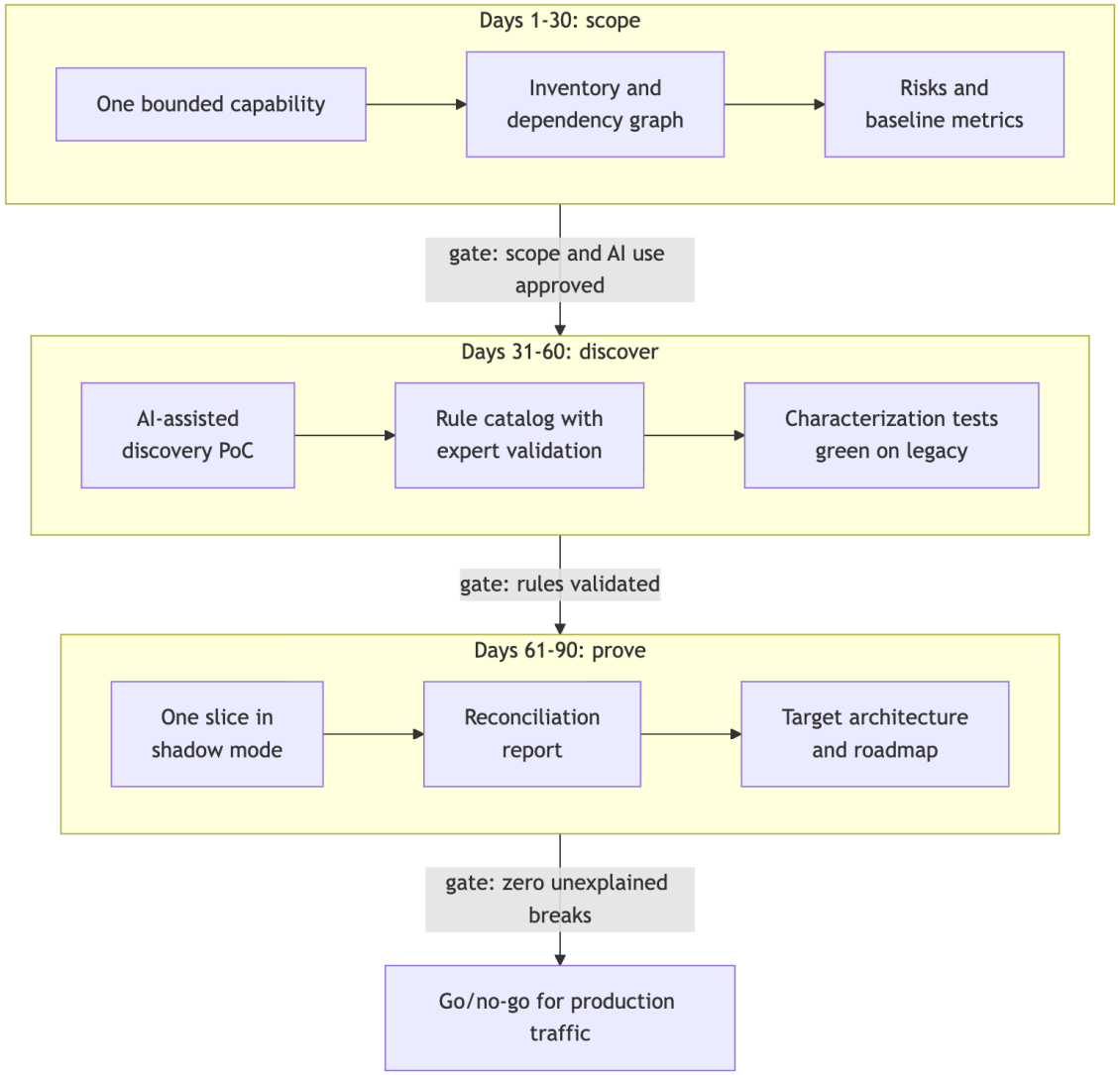
Testing and reconciliation

- **Golden master:** capture legacy outputs before changing anything
- **Record and replay** masked production transactions on both systems
- **Differential testing** field by field, tolerance normally zero for amounts
- **Property-based testing** for rounding, accrual, fee caps, FX
- **Reconcile at three levels:** record, account balance, general ledger per business date
- **Parallel run** over a full cycle with month-end
- **Operational tests:** failover, mid-batch restart, duplicates, replay

A first 90-day initiative

- **Days 1-30, scope:** one bounded capability, inventory, risks, baseline metrics
- **Days 31-60, discover:** AI discovery PoC, rule catalog, characterization tests
- **Days 61-90, prove:** one slice in shadow mode, reconciliation report, roadmap

Pick a capability **valuable enough to matter, bounded enough to finish**.



How to know it is working

Measure	How to measure
Discovery effort	Person-days per program or capability, with and without AI
Business rules validated	Confirmed by experts / candidates extracted
Rule extraction quality	Precision and recall on the evaluation set
Reconciliation discrepancies	Count and value of breaks per business date, by root cause
Change lead time	Request to production, before and after
Cost	Legacy plus new during coexistence; MIPS/MSU reduction

Open questions to settle early

- Which system is the **book of record** for each data element during coexistence?
- What is the **tolerance policy** for reconciliation differences, and who signs it off?
- Which **AI models and hosting** are approved by security, risk and procurement?
- What are the **regulatory notification** requirements for material ICT changes?
- What is the **decommissioning date** for each migrated component?

Key message

Turn undocumented legacy banking systems into **understandable, testable and incrementally modernizable** business capabilities, while preserving financial correctness and operational risk controls.

Resources

Repo (context, rule catalog, COBOL and Java examples):
github.com/wallaceespindola/banking-modernization-with-ai

Wallace Espindola, Solution Architect
GitHub: github.com/wallaceespindola
LinkedIn: linkedin.com/in/wallaceespindola
Speaker Deck: speakerdeck.com/wallacese

Thanks! Questions?